

Prescriptive Algorithms - Assignment 3

Lars Kerkhof (2082570)
Ralf Geerts (2101481)
Vera Beerepoot (2163112)
Yvette Garretsen (2100222)

Jheronimus Academy of Data Science
Prescriptive Algorithms (JM0100)

12 May 2026

Contents

1	Prediction Task	2
1.1	Feature Construction and RUL (Task 1.1.1)	2
1.2	Model, Validation, and Hyperparameters (Tasks 1.1.2 & 1.1.3)	2
1.3	Predictions for Dataset 2 (Task 1.2.1)	2
1.4	Comparison with Consultancy Predictions (Task 1.2.2)	3
2	Optimization Task	4
2.1	Genetic Algorithm (Task 2.1)	4
2.2	Optimization with Model-Based RULs (Task 2.2)	4
2.3	Comparison with Consultancy RULs (Task 2.3)	5

1 Prediction Task

1.1 Feature Construction and RUL (Task 1.1.1)

Dataset 1 is consistent with the NASA C-MAPSS benchmark (Saxena et al., 2008) and contains no missing values. Since the dataset contains full run-to-failure trajectories, we computed RUL as $RUL_{i,t} = \max_{t'}(\text{cycle}_{i,t'}) - \text{cycle}_{i,t}$, yielding $RUL = 0$ at the failure cycle.

Feature selection. We removed seven features that were constant across all 20,631 observations (`set1`, `sensor_val`{7, 8, 11, 14, 16, 20}), as they carry no predictive information. Additionally, we dropped `cycle` and the computed `max_cycles` column, because RUL is constructed using cycles, so retaining them would leak the target into the predictors. We then computed the Pearson correlation of each remaining feature with RUL. Three features (`set2`, `set3`, `sensor_val18`) showed $|r| < 0.15$; a baseline XGBoost model confirmed these had negligible feature importance and they were dropped. The final set consists of 14 predictors.

RUL clipping. Engines in early life show no measurable degradation, so sensor readings cannot reliably distinguish very high RUL values. Following Heimes (2008), we applied a piecewise-linear degradation model by capping the target at $R_{\max} = 125$ cycles, a commonly used value in C-MAPSS literature (Zheng et al., 2017). This intentionally focuses the model on the degradation-relevant range, which is also the range that matters most for the 30-day maintenance schedule.

1.2 Model, Validation, and Hyperparameters (Tasks 1.1.2 & 1.1.3)

XGBoost is suitable because its tree ensemble captures non-linear relationships, its sequential boosting procedure improves on earlier errors, and its regularization and sampling parameters help control overfitting. We split the data at the **engine level** (20 engines held out, `random_state=12345678`) to avoid leakage from correlated consecutive-cycle observations within the same engine. Within the training set we used 5-fold `GroupKFold` with `engine_id` as the grouping key.

We chose **RMSE** as the primary metric (interpretable in cycles, penalizes large errors), and report MAE and R^2 as diagnostics. Hyperparameter tuning used `RandomizedSearchCV` (50 iterations) covering all four required categories: *tree complexity* (`max_depth`, `min_child_weight`), *learning process* (`n_estimators`, `learning_rate`), *sampling* (`subsample`, `colsample_bytree`), and *regularization* (`reg_alpha`, `reg_lambda`, `gamma`). Random search was preferred over grid search for efficiency (Bergstra and Bengio, 2012).

Table 1 shows the incremental gains. RUL clipping accounted for the dominant improvement; hyperparameter tuning contributed a smaller additional gain. The final model achieves Test RMSE = 16.78, MAE = 12.77, $R^2 = 0.837$.

Configuration	Features	CV RMSE	Test RMSE	Test R^2
Baseline (no clipping)	17	42.83	–	–
After dropping low-importance features	14	42.73	–	–
+ RUL clipping ($R_{\max} = 125$)	14	19.46	17.05	0.832
+ Hyperparameter tuning	14	19.19	16.78	0.837

Table 1: Evaluation results showing incremental improvements.

1.3 Predictions for Dataset 2 (Task 1.2.1)

The same preprocessing was applied to dataset 2. For each of the 100 engines we selected the latest observed cycle (the current operational state). The final model was retrained on all training engines with the tuned hyperparameters and predictions were rounded to the nearest non-negative integer, yielding predicted RUL values from 7 to 122.

1.4 Comparison with Consultancy Predictions (Task 1.2.2)

We compared our model’s predictions against the consultancy’s estimates using four evaluation metrics. The **Pearson correlation** ($r = 0.908$, $p < 0.001$) confirms a strong agreement on the relative ranking of engine degradation. To evaluate systematic bias, we conducted a **paired t-test** alongside a **Wilcoxon signed-rank test**. Because our model’s predictions are explicitly capped at a threshold of 125 cycles, the resulting error distribution between the two models is inherently non-normal. Therefore, the Wilcoxon test serves as a necessary non-parametric robustness check, ensuring our conclusions regarding bias remain statistically valid without relying on the assumption of normality. Mean Absolute Error (MAE) was used to quantify the typical per-engine discrepancy.

The results (mean difference = -4.65 cycles, MAE = 14.61, RMSE = 21.46, paired t-test $t = -2.208$ with $p = 0.0295$, Wilcoxon $p = 0.0434$) indicate that while the models agree strongly on engine ranking, there is a small, statistically significant bias. Furthermore, the moderate gap between the RMSE and MAE indicates the presence of localized, larger deviations rather than a uniform spread of errors. Figure 1 elucidates the source of this bias: the models align closely for engines with a low RUL but diverge for high-RUL engines. Our predictions plateau near the designed clipping threshold of 125, whereas the consultancy’s predictions extend up to 182, suggesting no clipping was applied. Crucially, because the maintenance planning horizon is limited to $T = 30$ days, this divergence occurs entirely outside the actionable scheduling range. Consequently, the observed statistical bias reflects a conscious methodological design choice rather than a meaningful predictive disagreement, and it has no adverse impact on the downstream prescriptive optimization.

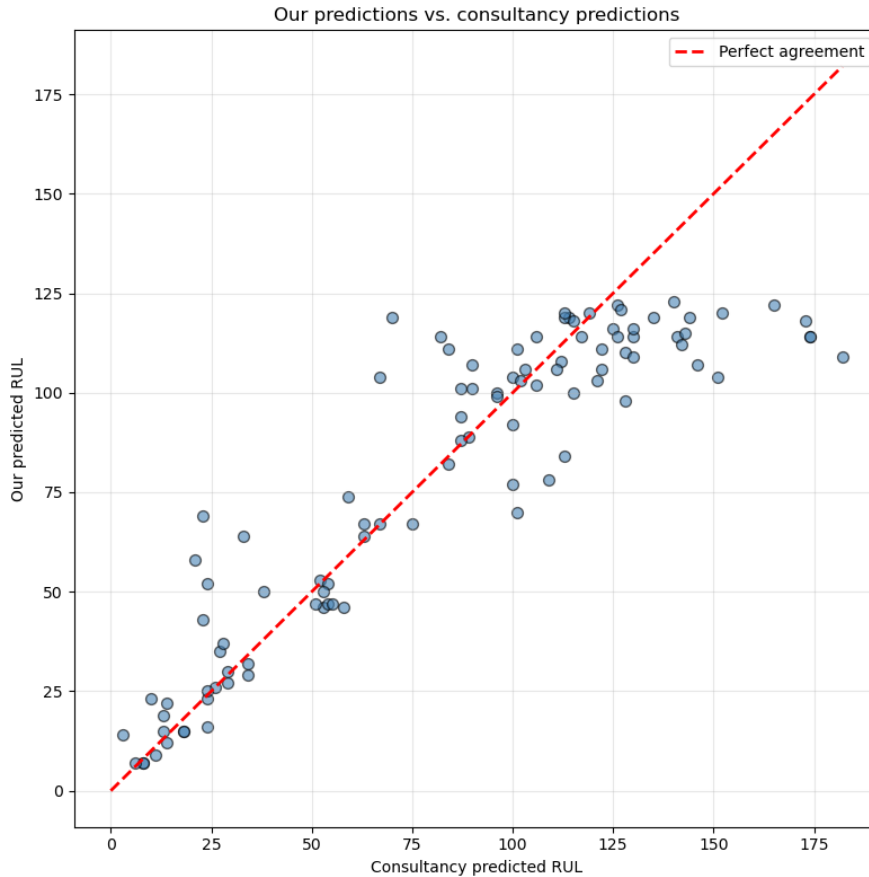


Figure 1: Our predictions vs. consultancy predictions. Red dashed line = perfect agreement.

2 Optimization Task

2.1 Genetic Algorithm (Task 2.1)

We developed a repair-based Genetic Algorithm (GA) to construct feasible maintenance schedules for engines with a predicted safety due date below 30. The current day is $t = 1$ and the planning horizon is $T = 30$. The objective is to minimize total penalty cost.

Individual representation. An individual represents a complete schedule for the candidate engines. Each gene corresponds to one engine and is encoded as (`team_code`, `start_day`). A `team_code` of 0 means that the engine is not scheduled. Otherwise, the code indicates one of the four teams. The end day is derived from the start day and the maintenance duration of the assigned team type.

Fitness function. The fitness value is the total penalty cost, so lower values are better. For maintained engines, penalties are accumulated from the day after the safety due date until maintenance completion. For unscheduled engines, penalties are accumulated until the end of the 30-day horizon.

Genetic operators. Parents are selected using tournament selection with tournament size 3. We use uniform crossover, where each engine-level gene is swapped between two parents with probability 0.5. Mutation is applied at the gene level: a selected gene is replaced by a newly generated random gene, meaning that the engine can become unscheduled or receive a new team and start day.

Infeasibility handling. After initialization, crossover, and mutation, individuals are repaired using `decode_and_repair`. The repair step enforces that teams do not overlap, engines are maintained at most once, and maintenance is completed within the planning horizon. If a proposed assignment is infeasible, the algorithm first tries to repair it using the same team, then alternative teams. If no feasible option exists, the engine remains unscheduled.

Parameter settings. The final GA settings were: population size 50, 65 generations, crossover probability 0.80, mutation probability 0.20, gene-level mutation probability 0.10, tournament size 3, and initial unscheduled probability 0.35. These settings kept the required 30-run experiments computationally feasible while still allowing improvement over generations.

Output. The GA returns a full 100-engine solution table. For maintained engines, the table reports team type, start day, end day, and penalty cost; for unmaintained engines, these fields remain empty and penalties are still calculated where relevant. The validation function recalculates all safety due dates, maintenance durations, team overlaps, horizon constraints, and penalty costs. The final check run produced a feasible schedule with total penalty cost 0.

2.2 Optimization with Model-Based RULs (Task 2.2)

The GA was first applied to the integer-valued RUL predictions from the XGBoost model. A single model-based GA run completed in 4.74 seconds and found a feasible zero-penalty schedule. For this RUL input, 18 engines had a safety due date within the 30-day planning horizon. In the best solution over the required 30 independent runs, all 18 candidate engines were scheduled for maintenance and the resulting total penalty cost was 0. The full 100-engine schedule, including team type, start day, end day, and engine-level penalty costs, is reported in the notebook output. The validation check confirmed that all maintenance jobs were completed within the planning horizon, no team-overlap constraints were violated, and all penalty costs were recalculated consistently. All individual GA runs stayed below the five-minute limit; the longest repeated run took 4.95 seconds.

Metric	Model-based RULs	Consultancy RULs
Best total penalty	0	55
Mean total penalty over 30 runs	85.33	382.93
Standard deviation	77.61	222.05
Worst total penalty	290	1216
Candidate engines	18	24
Maintained engines in best run	18	23
Feasible best run	Yes	Yes

Table 2: Optimization results over 30 GA runs for model-based and consultancy RUL inputs.

The 30-run experiment shows that the GA consistently improves substantially from its initial populations. For the model-based RULs, the average best-so-far fitness decreases sharply in the early generations and then flattens as the algorithm converges. This pattern suggests that the repair-based GA is able to quickly identify feasible low-penalty schedules.

2.3 Comparison with Consultancy RULs (Task 2.3)

The same GA settings and feasibility checks were then applied to the consultancy RUL predictions. This ensures that any difference in the optimization result is caused by the RUL input values rather than by different algorithmic settings. With the consultancy RULs, 24 engines had a safety due date within the planning horizon, compared to 18 engines under the model-based RULs. Thus, the consultancy input created a more constrained scheduling problem with more urgent engines competing for the same four maintenance teams.

The best consultancy-based solution achieved a total penalty cost of 55. It scheduled 23 of the 24 candidate engines and left one candidate engine unscheduled. Several maintained engines also incurred small delay penalties because maintenance finished after their safety due date. The best model-based solution, in contrast, achieved a penalty of 0 because all 18 candidate engines could be maintained on time. At the engine-decision level, the two best schedules made the same maintenance decision for 93 engines and differed for 7 engines.

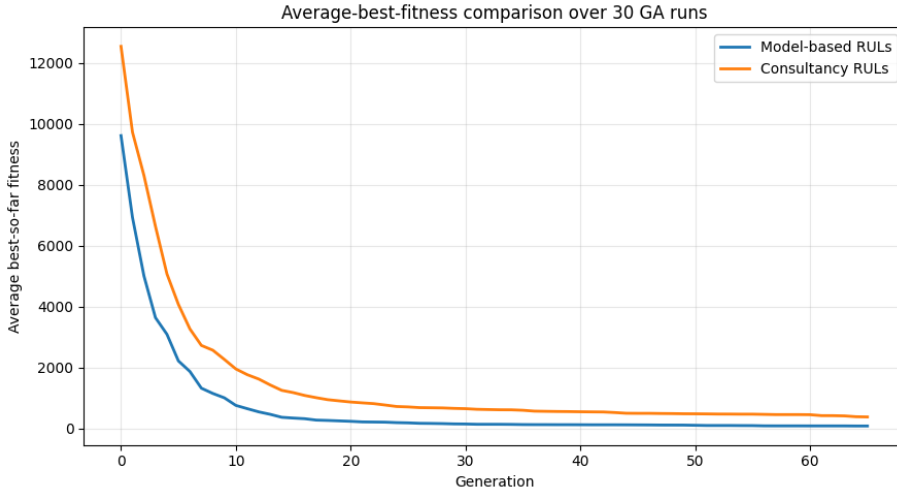


Figure 2: Average-best-fitness over 30 GA runs for the model-based and consultancy RUL inputs.

Overall, the consultancy-based optimization produced higher and more variable penalty costs. This difference is consistent with the prediction comparison in Section 1.4: although the two RUL sources agree strongly in ranking, the consultancy predictions place more engines below the 30-day scheduling threshold. Therefore, the consultancy input increases the number of urgent maintenance jobs and makes it harder for the GA to construct a zero-penalty schedule. Both best solutions were feasible, so the observed difference reflects the different RUL inputs rather than constraint violations.

References

- Bergstra, J., & Bengio, Y. (2012). Random search for hyper-parameter optimization. *Journal of Machine Learning Research*, 13, 281–305.
- Heimes, F. O. (2008). Recurrent neural networks for remaining useful life estimation. In *2008 International Conference on Prognostics and Health Management* (pp. 1–6). IEEE.
- Saxena, A., Goebel, K., Simon, D., & Eklund, N. (2008). Damage propagation modeling for aircraft engine run-to-failure simulation. In *2008 International Conference on Prognostics and Health Management* (pp. 1–9). IEEE.
- Zheng, S., Ristovski, K., Farahat, A., & Gupta, C. (2017). Long short-term memory network for remaining useful life estimation. In *2017 IEEE International Conference on Prognostics and Health Management (ICPHM)* (pp. 88–95). IEEE.